```
import heapq
import os

class HuffmanNode:
    def __init__(self, char=None, freq=0, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    # For heap comparison
    def __lt__(self, other):
        return self.freq < other.freq


from collections import defaultdict

def calculate_frequencies(file_path):
    """Calculate character frequencies in a text file"""
    frequencies = defaultdict(int)
    with open(file_path, 'r') as file:
        for line in file:
            for char in line.lower():
                if char.isalpha():
                    frequencies[char] += 1
    return frequencies

def build_huffman_tree(frequencies):
    heap = []
    for char, freq in frequencies.items():
        heapq.heappush(heap, HuffmanNode(char=char, freq=freq))

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(freq=left.freq + right.freq, left=left, right=right)
        heapq.heappush(heap, merged)

    return heapq.heappop(heap) if heap else None

def generate_codes(node, current_code="", codes=None):
    if codes is None:
        codes = {}

    if node is None:
        return

    if node.char is not None:  # Leaf node
        codes[node.char] = current_code
        return
```

```
        generate_codes(node.left, current_code + "0", codes)
        generate_codes(node.right, current_code + "1", codes)

        return codes

    def encode_file(input_path, output_path, codes):
        with open(input_path, 'r') as input_file, open(output_path, 'w') as output_file:
            for line in input_file:
                for char in line.lower():
                    if char.isalpha() and char in codes:
                        output_file.write(codes[char])

    def decode_file(encoded_path, output_path, huffman_tree):
        with open(encoded_path, 'r') as encoded_file, open(output_path, 'w') as output_file:
            current_node = huffman_tree
            bits = encoded_file.read()

            for bit in bits:
                if bit == '0':
                    current_node = current_node.left
                else:
                    current_node = current_node.right

                if current_node.char is not None:
                    output_file.write(current_node.char)
                    current_node = huffman_tree

    def verify_files(original_path, decoded_path):
        with open(original_path, 'r') as original_file, open(decoded_path, 'r') as decoded_file:
            original_content = original_file.read().lower()
            decoded_content = decoded_file.read()

            original_alpha = ''.join(c for c in original_content if c.isalpha())
            decoded_alpha = ''.join(c for c in decoded_content if c.isalpha())

            return original_alpha == decoded_alpha

    def main():
        input_file = "input.txt"
        encoded_file = "encoded.txt"
        decoded_file = "decoded.txt"

        frequencies = calculate_frequencies(input_file)
        print("Character frequencies:", frequencies)

        huffman_tree = build_huffman_tree(frequencies)

        codes = generate_codes(huffman_tree)
        print("Huffman codes:", codes)

        encode_file(input_file, encoded_file, codes)

    if __name__ == "__main__":
```

```
main()
```

```
Character frequencies: defaultdict(<class 'int'>, {'a': 3, 'b': 2, 'c': 2})
Huffman codes: {'a': '0', 'b': '10', 'c': '11'}
```